Machine Learning — Challenge 1
Reinforcement Learning (DQN) for Atari

Prof. Carlos Andrés Sierra, M.Sc.
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Overview

This document describes `Challenge 1` for the **Machine Learning** course: an individual-group task focused on designing, training and analysing *Deep Q-Network (DQN)* agents to solve a selected Atari environment from the Arcade Learning Environment (ALE).

The objective is to produce a reproducible implementation, conduct a systematic hyperparameter study, and deliver a concise *scientific report* in *IEEE format* justifying the final design choices.

Challenge objective

Students will implement a `DQN-based reinforcement learning agent`, execute multiple controlled experiments, and identify the configuration that yields the best empirical performance on a chosen ALE environment. The work must emphasise experimental rigour: reproducible code, clear logging, principled hyperparameter sweeps, and a scientific justification linking observed behaviour to algorithmic design choices.

Available environments (one per group)

Each group (1 — 8) selects exactly one environment from the list below. The environments are intentionally challenging (sparse rewards, long-horizon planning, or complex state dynamics). Use the ALE environment identifier when configuring the agent (for example: `ALE/MontezumaRevenge-v5`).

1. Group 1 — **ALE/MontezumaRevenge-v5**: very sparse rewards, requires long-term planning and memory of visited rooms.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

2. Group 2 — **ALE/Pitfall-v5**: sparse/hidden rewards and delicate action sequences to reach high scores.
3. Group 3 — **ALE/PrivateEye-v5**: complex objectives and sparse rewards across long episodes.
4. Group 4 — **ALE/Gravitar-v5**: challenging physics and navigation; requires precise control.
5. Group 5 — **ALE/Solaris-v5**: multi-stage dynamics and varied objectives across levels.
6. Group 6 — **ALE/Venture-v5**: high difficulty early, sparse positive signals in a noisy action space.
7. Group 7 — **ALE/MsPacman-v5**: complex moving objects, partial observability and reactive planning.
8. Group 8 — **ALE/Phoenix-v5**: fast action dynamics and non-trivial reward structure.

Deliverables

Each group must provide the following deliverables by the deadline specified by the instructor:

- **Repository folder:** A GitHub repository with a single folder for this challenge (e.g., `challenge1__<group_id>/`). The folder must include all source code, a `README.md` with reproducible run instructions, and logging artifacts (sample `TensorBoard logs`). The repository must contain a short `pyproject.toml` describing the environment.
- **Scientific report (IEEE format):** A single, self-contained paper in IEEE conference format (PDF). The paper must present experimental methodology, hyperparameter search strategy, quantitative results, ablation analyses, discussion and conclusions. The paper is limited to 6 pages (excluding references) and must be reproducible from the repository instructions.
- **Short video (<=10 minutes):** A brief video (no more than 10 minutes) in which the students succinctly explain the proposed solution and experimental results, and demonstrate the trained agent playing the selected environment. Provide a stable external link (YouTube, Vimeo, or institutional storage) in the README. The README must include timestamps and a short caption describing the contents of the video.

Expectations and experimental protocol

Groups must follow a rigorous experimental protocol. At minimum, the following elements are required:

1. Baseline implementation of a DQN agent with standard preprocessing (grayscale, frame resize to 84×84, frame stacking of 4 frames, frame-skip 4).
2. Reproducible training runs: fix random seeds, record all seeds used, and provide Docker/virtualenv instructions when possible.
3. Hyperparameter search: perform controlled sweeps over a set of hyperparameters (see suggested search space below). Each experimental condition must be repeated for at least three independent random seeds to report variance.
4. Logging and metrics: record per-episode reward, rolling mean reward (100-episode window), epsilon schedule, and training loss. Provide final aggregated tables and plots in the report.
5. Analysis: provide a concise justification for the selected final configuration, supported by empirical evidence (learning curves, ablation where appropriate) and discussion of failure modes.

Suggested hyperparameter search space

The instructor encourages exploration: the final chosen parameters should not merely copy published defaults. As a starting point, consider the following ranges (each group chooses a design for how to explore these ranges systematically):

- Learning rate (α): 5×10^{-5}, 1×10^{-4}, 2×10^{-4}.
- Replay buffer size: 50000; 100000; 200000.
- Learning starts: 10000; 25000; 50000.
- Batch size: 32; 64; 128.
- Discount factor (γ): 0.99; 0.995.
- Train frequency (env steps per update): 4; 8.
- Target network update interval: 1000; 2000; 5000.
- Exploration schedule: exploration fraction 0.15 — 0.30; final epsilon 0.001 — 0.02.

Groups must explain their exploration strategy (random grid search, Bayesian optimisation, successive halving, etc.) and justify the computational budget assigned to each experiment.

Evaluation criteria

Submissions will be evaluated according to the following aspects:

- **Reproducibility (25%):** clarity of instructions, ability to reproduce reported runs, presence of seed logging and environment specifications.
- **Experimental rigor (35%):** completeness of hyperparameter exploration, correct reporting of variance, and methodological soundness.
- **Performance and analysis (30%):** empirical performance of the final agent, appropriateness of comparisons, and depth of analysis explaining why selected parameters worked.
- **Presentation and writing (10%):** quality of the IEEE paper (structure, clarity, figures, tables and citations).

Submission format and deadlines

Submit a single *GitHub repository* link and the *IEEE-format PDF* to the course submission portal. The repository must contain a top-level folder named `challenge1__<group_id>` and a `README.md` with exact commands to reproduce the best reported run. The IEEE PDF must be uploaded as `challenge1__group_id_paper.pdf`.

Notes on scope and recommended computational budget

These environments are computationally demanding. Each group should plan experiments according to available compute. When compute is limited, document the constraints and prioritise systematic, lower-budget experiments that focus on understanding trends rather than exhaustive sweeps.

References and further reading

Provide standard references for DQN, replay buffers, target networks, and exploration scheduling in the final paper. Suggested starting points include the original DQN paper and modern survey articles on deep reinforcement learning; however, groups are expected to seek up-to-date sources and cite them appropriately in IEEE style.

This challenge is designed to develop experimental discipline and a deep understanding of how algorithmic choices interact with environment complexity. Good experimental design and clear explanation of results are more highly valued than marginal performance gains achieved through ad-hoc tuning.